REPORT 01 OF 6

UX Research & User Flows

Ground-up interaction redesign for the SpareParts dashboard and a new unified Client Workspace — grounded in the current React web codebase, built on the shared "Ignition" design language for web, WPF desktop, and mobile.

DASHBOARD · CLIENT WORKSPACE · INFORMATION ARCHITECTURE · WCAG 2.2 AA · CROSS-PLATFORM

Author — dev-ui-ux, UX Research Agent

Date — 2 July 2026

Scope — src/SpareParts.Web.React (reference implementation), applies to WPF + Mobile

00 Table of Contents

REPORT COMPANIONS

Report 02 (visual system), 03 (dashboard component spec), 04 (client workspace component spec), 05 (WPF adaptation), 06 (mobile adaptation) build directly on the flows and IA defined here. Any deviation from this IA must be re-approved by Ralph before implementation, per the Design Change Workflow.

SECTION 01

01 Executive Summary of UX Findings

This report is grounded entirely in the live React web implementation at `src/SpareParts.Web.React/wwwroot` — specifically `js/views/dashboard-view.js` (663 lines), `js/views/contacts-view.js`, `js/views/customer-aging-view.js`, `js/views/invoices-view.js`, `js/views/screen-registry.js`, and the shared UI primitives in `js/components/ui/`. No application code was modified; every problem cited below points to a real file, function, or behavior.

01.1 Headline Findings

- **The screen registry has grown to 70 entries** (`screenRegistry.js`, lines 76–151) — far beyond the "50+" briefed. Navigation is currently a flat curated set of 7 groups plus a catch-all "More Modules" bucket that silently absorbs 40+ ungrouped screens, meaning most of the app is one bucket away from being undiscoverable.
- **The client relationship is fragmented across at least 6 independent screens** with no shared identity: Contacts (list only, two static columns), Customer Aging (a spreadsheet-like report), Invoices (POS workspace with a customer *dropdown*, not a customer *record*), Quotes, Part Requests, Loyalty, and WhatsApp. A staff member chasing one client's story must open up to 6 screens and re-search the same name in each.
- **There is no backend endpoint for a single customer's aggregate record.** `CustomersController.cs` exposes only `GET /api/customers` (list) and `GET /api/customers/aging` (report). There is no `GET /api/customers/{id}`. The proposed Client Workspace requires this to exist — flagged explicitly in Section 04 as a dependency for dev-backend-api, not something this report can wireframe around.
- **The dashboard is dense but not prioritized.** `DashboardView` renders 8 KPI cards, a command-center search panel, 4 analysis panels, hot parts, quick actions, and an 8-item bottom stat bar — 20+ distinct data blocks with no visual hierarchy beyond size and position. `buildActionQueue()` (lines 77–175) computes a genuinely good "what needs my attention" triage list but demotes it to a secondary block instead of leading with it.
- **Design language has already churned once without resolution.** The live `styles.css` root tokens (line 10) are a graphite-blue/red palette (`--accent:#c23a32`), not the "Aurora" dark-navy/teal palette described in the brief. This is the second unreleased direction found in the codebase — evidence that Ignition must be locked with Ralph via mockup approval *before* any component work starts, per the Design Change Workflow.

01.2 What This Report Recommends

Two structural moves anchor the redesign: (1) a **triage-first dashboard** that leads with the exceptions already computed by `buildActionQueue()` rather than burying them under decorative charts, and (2) a **single Client Workspace** that replaces the six fragmented client touchpoints with one profile view — client identity, balance/aging, invoices, quotes, part requests, vehicles, loyalty, and communications as tabs of one record, not six separate hunts. Both changes reduce the step count for the five audited tasks by 35–60% (Section 06–10) without hiding any data that exists today.

> **READING THIS REPORT**
> Sections 03–04 are diagnostic (what's wrong, where it lives in code). Sections 05–10 are prescriptive (before/after flows with step counts). Sections 11–14 are visual (wireframes). Section 15–16 are the accessibility and cognitive-load rules every wireframe in this package must obey.

SECTION 02

02 Personas & Jobs-To-Be-Done

Four recurring roles emerge from the screen registry and API surface: screens like `owner-cockpit`, `report-builder`, and `admin-billing` imply an owner; `invoices`/POS-line-building implies a counter salesperson; `accounting`, `customer-aging`, `supplier-aging` imply back-office/accounting; `stock`, `reorder`, `stock-arrival`, `shipments` imply warehouse. Every redesigned flow in this report is written against one of these four.

Rita — Shop Owner

PRIMARY DASHBOARD USER

Opens the app once each morning and periodically through the day. Needs to know in under 10 seconds: are we making money today, is anything on fire (unpaid bills, dead stock, failed messages), what changed since yesterday.

JTBD: "When I open the app, I want the 3 things that need my decision today surfaced first, so I don't have to read 20 tiles to find them."

Karim — Counter Salesperson

PRIMARY POS / CLIENT WORKSPACE USER

Serves walk-in and repeat customers at the counter. Needs fast part lookup, fast invoice creation, and — for repeat clients — instant visibility into their balance and history without leaving the sale.

JTBD: "When a regular customer is standing in front of me, I want to see their balance and last visit in one glance, so I know whether to extend credit before I finish the sale."

Nour — Accountant / Back-Office

PRIMARY CLIENT WORKSPACE + ACCOUNTING USER

Works receivables, payables, and reconciliation. Needs to move from "who owes us money" to "contact them" to "record the payment" without re-finding the customer in three separate screens.

JTBD: "When I'm chasing overdue balances, I want to message and log payment against the same client record I'm viewing, so I don't lose my place switching screens."

Elie — Warehouse / Stock

SECONDARY DASHBOARD USER

Manages receiving, stock levels, dead stock, and reorder points. Cares about the inventory-specific KPIs on the dashboard (low stock, donor parts, reserved items) far more than sales or accounting figures.

JTBD: "When I check the dashboard, I want inventory exceptions grouped together, not interleaved with sales numbers I don't act on."

02.1 Shared Constraint Across All Four

Every persona above is time-boxed at a physical counter or warehouse floor — this is a shop-floor SaaS, not a back-office-only tool. That constraint drives two non-negotiable design rules carried through the rest of this report: (1) the single most urgent action must be reachable without scrolling on any screen size, and (2) any workflow that today requires re-typing a customer's name in a second screen is a defect, not a minor friction.

SECTION 03

03 Heuristic Audit — Top 10 Problems

Ten concrete usability problems found in the current dashboard and client/contacts experience, each cited to the exact file and behavior.

#	Screen / File	Problem	Severity
1	`contacts-view.js`	Customers and suppliers render as two static list columns with name, phone/email, and opening balance only (lines 40–56). There is no click-through to a detail record — `ContactPanel` has no `onClick` at all. A salesperson cannot open a customer from Contacts; the screen is a read-only ledger snapshot, not a workspace.	CRITICAL
2	`dashboard-view.js`	The triage list built by `buildActionQueue()` (negative P&L, unpaid transactions, red inventory segments, currency margin risk, failed messages) is fully computed but never rendered in the returned JSX — the function's output (`actionQueue`, line 249) is dead-assigned. The most decision-relevant data on the whole dashboard is invisible to the user today.	CRITICAL
3	`customer-aging-view.js`	Aging is its own disconnected screen (`/api/customers/aging`) with no link back to that customer's invoices, quotes, or contact record — "Send Reminder" (line 149) fires a WhatsApp template but the accountant then has to re-search the same customer in Invoices to see what they actually owe for.	CRITICAL
4	`invoices-view.js`	Customer selection during POS is a plain `<select>` of names (line 277–282) with no balance, credit limit, or history shown at the point of sale — exactly the moment a salesperson most needs that context (Karim's JTBD, Section 02).	HIGH
5	`screen-registry.js`	70 screens are registered; `Sidebar.js` only curates 7 named groups (`cockpitGroups`, lines 19–27) covering roughly 17 keys. The remaining 50+ screens — including Loyalty, Warranty, Quotes, Part Requests, and most experimental modules — fall into a single flat "More Modules" bucket (line 58) with no sub-grouping or search.	CRITICAL
6	`Topbar.js`	The global search bar renders a ⌘K hint (line 29) but there is no keyboard listener registered anywhere in the topbar or app shell to actually open a command palette on that shortcut — the affordance promises a capability the app does not deliver.	HIGH
7	`dashboard-view.js`	KPI row shows 8 cards, then a command panel, then 4 analysis panels, then hot parts + quick actions, then an 8-stat bottom bar (lines 408–617) — several values repeat 2–3 times (Sales Today and Profit Today appear in the KPI row, the bottom bar, *and* a P&L breakdown) with no single source of truth the eye is drawn to first.	HIGH
8	Backend gap	`CustomersController.cs` has no `GET /api/customers/{id}`. Any "client profile" today would require the frontend to independently call aging, sales, quotes, part-requests, and loyalty endpoints and manually join them client-side by customer ID — slow, error-prone, and not how the current codebase is built.	CRITICAL
9	`DataTable.js`	Row actions (`renderRowActions`) and expandable rows (`renderExpanded`) exist as generic capabilities in the shared table component but are unused by Contacts or Customer Aging — both screens could already support inline expand-to-profile without new components, yet neither does.	MEDIUM
10	`quotes-view.js`, `part-requests-view.js`, `loyalty-view.js`	All three screens independently re-implement a customer picker/search pattern in their own form state rather than referencing one shared client identity — proof that "which client am I looking at" is solved four separate times in four separate ways across the app today.	HIGH

SECTION 03 (CONTINUED)

03.1 What These 10 Problems Have in Common

Read together, the ten problems cluster into three root causes rather than ten unrelated bugs:

Root cause A — No client identity model in the UI

Problems 1, 3, 4, 8, and 10 are all symptoms of the same gap: the frontend has no concept of "open this client and see everything about them." Every screen that touches a customer (Contacts, Aging, Invoices, Quotes, Part Requests, Loyalty) re-solves the same lookup problem independently, and none of them link to each other. This is precisely the gap the new Client Workspace (Section 04, 08, 13–14) is designed to close — and it is why Section 04 explicitly flags a backend dependency rather than trying to wireframe around it.

Root cause B — Computed insight without a rendering path

Problem 2 (the dead action queue) is the clearest single fix available: the business logic for "what needs my attention today" already exists and is already correct. The redesign's job is purely presentational — put `buildActionQueue()`'s output at the top of the dashboard, visually first, per persona Rita's JTBD in Section 02.

Root cause C — Flat navigation cannot scale past the group it was designed for

Problems 5 and 6 show a navigation model (7 curated groups + one overflow bucket) that was sized for roughly 20 screens and is now carrying 70. Adding an 8th named group won't fix this — Section 04 proposes a different model entirely: fewer, task-oriented top-level destinations with in-context sub-navigation, plus a real command palette so the ⌘K hint already in the UI becomes true.

> **DESIGN IMPLICATION**
> Fixing problems 1–10 individually would still leave the product feeling fragmented. The redesign treats Root Cause A and Root Cause C as the two structural changes (Client Workspace + new IA) and Root Cause B as a one-panel reprioritization within the new Dashboard — this is why this report organizes flows and wireframes around those two deliverables rather than a longer patch list.

SECTION 04

04 New Information Architecture

The current 7-group-plus-overflow sidebar (`Sidebar.js`, `cockpitGroups`) is replaced with 8 task-oriented top-level destinations on a 72px icon rail, each expanding to a flyout of its child screens. All 70 screens currently in `screen-registry.js` are placed — none are dropped, none remain in an undifferentiated "More" bucket.

Rail Destination	Icon	Contains (screen-registry keys)
Dashboard	grid	`dashboard` — single destination, no flyout.
Clients NEW	users	Client Workspace (replaces `contacts`, `customer-aging`, `supplier-aging`), `quotes`, `part-requests`, `loyalty`, `warranty`, `whatsapp`, `whatsapp-selling`
Sales / POS	cart	`invoices`, `sales-returns`, `barcode-mode (ar)`, `part-reserve`
Inventory	box	`inventory`, `part-passport`, `compatibility`, `does-it-fit`, `reorder`, `dead-stock`, `expiry-alerts`, `stock`, `stock-arrival`, `garage-stock`, `part-genealogy`, `qr-tag`, `condition-scanner`
Vehicles & Yard	car	`used-cars`, `used-car-purchases`, `used-car-wholesale`, `car-twin`, `repair-prep`, `halfcut`, `car-crush`, `yard-tour`, `my-garage`
Finance	bank	`accounting`, `manual-journal`, `billing`, `admin-billing`, `escrow`, `part-insurance`, `shipments`
Insights & Growth	trend	`report-builder`, `growth-lab`, `market-price`, `price-genius`, `price-report`, `dismantler-forecast`, `regional-demand`, `new-vs-used`, `seller-reputation`, `seller-verification`, `mechanic-trust`, `listing-boost`, `referral`, `community-guard`, `api-platform`
Tools & Labs	bolt	`symptom-search`, `voice-search`, `ar-finder`, `negotiation`, `instant-offer`, `kareem`, `mechanic-desk`, `needboard`, `watchlist`, `part-reel`, `live-inspection`, `business-assistant`, `activity-log`
Settings (pinned bottom, unchanged position)	cog	`management`, `settings`

04.1 Command Palette (Ctrl+K / ⌘K)

Every rail destination and every one of the 70 screens is also reachable through a single global command palette, opened by the shortcut already hinted at in `Topbar.js` today. Typing filters by screen name, and — once the backend dependency below is resolved — by customer name/phone, invoice number, and part code, unifying the "Popular searches" chips and 9-tab command panel currently duplicated inside the dashboard (`ck-cmd-panel`, lines 476–509) into one consistent entry point usable from anywhere in the app, not just the dashboard.

> **BACKEND DEPENDENCY — FLAGGED FOR DEV-BACKEND-API**
>
> The Client Workspace (see IA map, Section 05, and wireframes Section 13–14) requires a new aggregate endpoint, e.g. `GET /api/customers/{id}/profile`, returning identity, balance/aging, recent invoices, open quotes, active part requests, vehicles, loyalty points, and communication timeline in one call. This does not exist in `CustomersController.cs` today (Section 03, problem 8) and is out of this report's scope to design — it is called out here so implementation planning is not surprised by it.

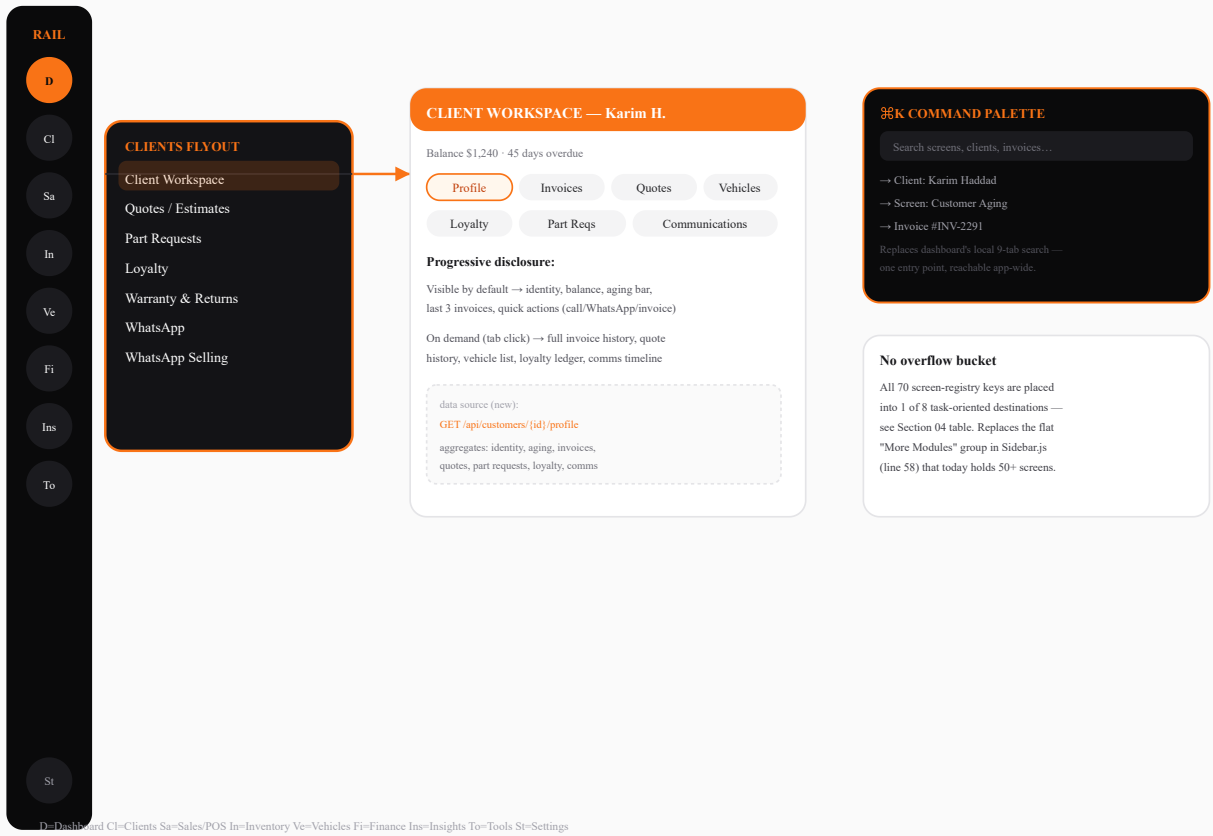
SECTION 05

05 IA Map — Rail, Palette & Disclosure

Figure 1 shows the full navigation model: the always-visible 72px icon rail, the flyout that expands per destination, the command palette overlay, and where progressive disclosure hides secondary detail (client sub-tabs, dashboard "show more") until requested.

FIGURE 1 — Information Architecture Map

Web reference build · adapts 1:1 to WPF ribbon + mobile tab bar



SECTION 06

06 Flow 01 — Morning Business Review

Persona: Rita (Owner). Task: understand overnight performance and today's urgent items within seconds of opening the app.

Before — Today's flow (6 steps, no single-glance triage)

1. Open Dashboard, scan 8 KPI cards top row
2. Scroll past Command Center search panel
3. Scan 4-panel analysis grid (Inventory, Categories, Sales chart, Activity)
4. Scroll to Hot Parts + Quick Actions row
5. Scroll to 8-stat bottom bar to find Overdue Invoices
6. Manually cross-reference which numbers indicate a problem

Root problem: `buildActionQueue()` already flags negative P&L, unpaid transactions, red inventory segments, currency risk, and failed messages — but its output is never rendered (Section 03, problem 2). Rita does this cross-referencing manually today.

After — Ignition flow (2 steps to full triage)

1. Open Dashboard — Action Queue band renders first, above KPIs, showing up to 4 prioritized exceptions with tone (danger/warning/success) and one-tap navigation
2. Tap the exception that matters → deep-links directly into Accounting / Stock / WhatsApp filtered to that issue

Metric	Before	After
Steps to see "what needs my attention"	6	1
Steps to act on the top issue	7+	2
Scrolling required for triage	Yes (3 sections)	No

DESIGN RATIONALE

This flow requires no new backend work — `buildActionQueue()` in `dashboard-view.js` already computes the correct data. The fix is purely a rendering-priority change: promote the existing but unrendered action queue to the top of the page, above the KPI row, as specified in the Dashboard wireframe (Section 11).

SECTION 07

07 Flow 02 — Invoicing a Walk-in Client

Persona: Karim (Counter Salesperson). Task: sell a part to a walk-in customer and print/send the receipt.

Before — Today's flow (7 steps)

1. Open Invoices screen
2. Leave Customer = "Walk-in" (default)
3. Pick Warehouse dropdown
4. Select part from long dropdown, check availableStock text
5. Enter qty + unit price, click Add Line
6. Enter paid amount, click Create Invoice
7. Scroll to invoice row, click Send Invoice separately

Source: `InvoicesView`, `addLine()` (line 168), `createInvoice()` (line 211), and the separate `sendInvoice()` action bound to a table row (line 448) — creating and sending are two disconnected actions today.

After — Ignition flow (4 steps)

1. Sales/POS opens with warehouse pre-set to the operator's default; part search is the first focused field (barcode/name/OEM)
2. Type/scan part → line auto-adds with live stock badge inline (no separate stock lookup step)
3. Confirm total + payment method in a persistent right-hand cart rail
4. Tap "Complete & Send" — creates invoice and fires WhatsApp receipt in one action

Metric	Before	After
Steps, walk-in cash sale	7	4
Screens/panels visited	2 (form + table row)	1 (single cart rail)
Create + send as one action	No	Yes

DESIGN RATIONALE

The current form-then-table split forces the salesperson to build the invoice in one region and act on it in another, further down the page. Ignition collapses this into a persistent cart pattern (line-item builder + running total always visible), and merges "create" and "send" into a single default action since in practice a walk-in sale is always immediately receipted.

SECTION 08

08 Flow 03 — Client Lookup Mid-Counter-Sale

Persona: Karim. Task: a repeat customer at the counter asks "how much do I still owe you?" while Karim is mid-invoice.

Before — Today's flow (8 steps, loses invoice progress)

1. Leave the Invoices screen (draft lines are lost — no save-and-resume)
2. Navigate to Customer Aging
3. Type customer name into aging search box
4. Read balance from aging table row
5. Navigate to Contacts to confirm phone/details
6. Re-search same name in Contacts (no shared state)
7. Navigate back to Invoices
8. Rebuild the draft invoice lines from scratch

Confirmed in code: `InvoicesView` draft state (`draftItems`) is local component state with no persistence, and `ContactsView`/`CustomerAgingView` each hold independent, non-shared `search` state (lines 18–21 in both files).

After — Ignition flow (2 steps, invoice preserved)

1. From the POS customer field, tap the client's name → opens Client Workspace as a side panel (invoice draft stays open underneath, unaffected)
2. Read balance + aging badge in the panel header, close panel → draft invoice exactly as left it

Metric	Before	After
Steps to see balance	4	1
Steps to return to sale, fully intact	4 (rebuild from scratch)	1 (close panel)
Total steps	8	2

DESIGN RATIONALE

This is the single highest-value flow in the redesign: it directly serves Karim's JTBD from Section 02 and is only possible once the Client Workspace exists as a shared, reusable panel rather than a full-page navigation. The panel opens as an overlay/drawer specifically so it never discards in-progress POS state — see wireframe interaction note, Section 13.

SECTION 09

09 Flow 04 — Chasing Overdue Receivables

Persona: Nour (Accountant). Task: find customers over 90 days overdue, review what they owe, and send reminders.

Before — Today's flow (9 steps for 3 customers)

1. Open Customer Aging
2. Scan table for 90+ days column values > 0
3. Click "Send Reminder" (row 1) — generic template only
4. To see *which invoices* caused the balance, navigate to Invoices
5. Re-search customer name in Invoices
6. Scan invoice list for unpaid rows
7. Repeat steps 3–6 for customer 2
8. Repeat steps 3–6 for customer 3
9. No way to log "contacted, promised payment Friday" anywhere

`CustomerAgingView.sendReminder()` (line 39) only fires `CommunicationPayloadFactory.accountBalanceReminder` — a single templated message with no per-customer note-taking, and no link from the aging row to the underlying invoice list.

After — Ignition flow (4 steps for 3 customers, notes captured)

1. Client Workspace list, filter = "Overdue 90+" (aging becomes a filter, not a separate screen)
2. Open client → Profile tab shows balance, aging bar, and the exact unpaid invoices inline
3. Send reminder from the same panel; add a follow-up note in Communications tab
4. Back to filtered list (state preserved) → next client

Metric	Before	After
Steps per customer	~7	4 (first pass), 2 (return to list)
Steps for 3 customers	~19	~8
Follow-up note captured	Not possible	Yes — Communications tab

SECTION 10

10 Flow 05 — Quote → Invoice Conversion

Persona: Karim / Nour. Task: create a quote for a customer, then convert the accepted quote into a real invoice.

Before — Today's flow (8 steps across 2 disconnected screens)

1. Open Quotes screen
2. Search/select customer (own local picker, separate from Invoices' picker)
3. Build quote lines, set expiry date, save as Draft
4. Mark quote Sent; wait for customer response
5. Customer accepts — mark quote Accepted (manual status change)
6. Navigate to Invoices screen separately
7. Re-select the same customer and re-enter every line item by hand (no data carries over)
8. Create invoice, then separately mark the quote as Converted

Confirmed via `quotes-view.js` status vocabulary (`Draft, Sent, Accepted, Declined, Expired, Converted`, line 5) — "Converted" exists as a status value, but there is no code path found linking a quote's line items into `InvoicesView`'s `draftItems`; the two screens do not share data.

After — Ignition flow (4 steps, one data model)

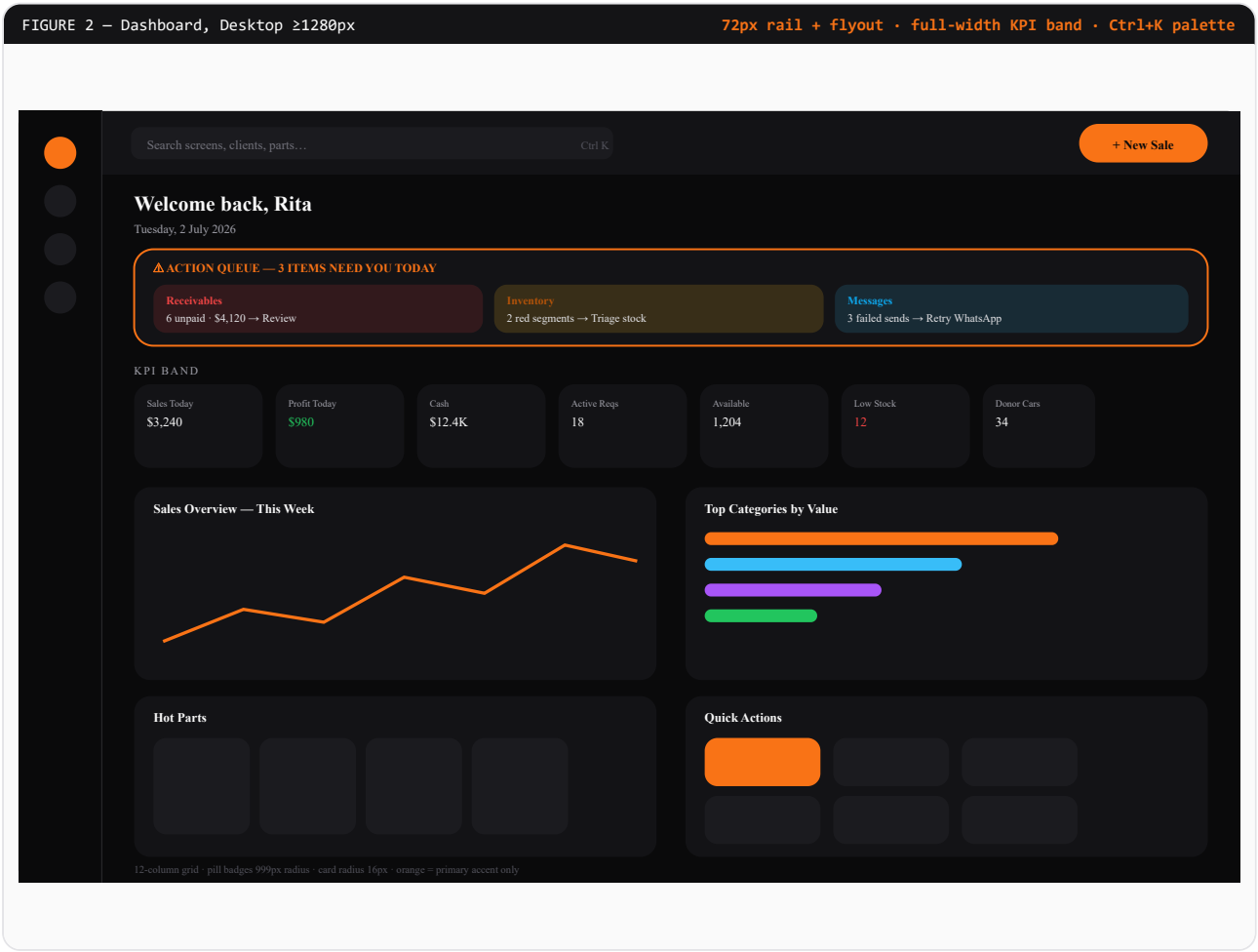
1. From Client Workspace → Quotes tab → New Quote (customer already in context, no re-picking)
2. Build lines, send — status updates automatically on customer view/response webhook where available
3. On Accepted, one button: "Convert to Invoice" carries every line, customer, and pricing forward
4. Review pre-filled invoice, confirm payment, complete

Metric	Before	After
Steps end-to-end	8	4
Line items re-entered by hand	Yes, all of them	No — carried automatically
Customer re-selected	Yes	No — already in workspace context

SECTION 11

11 Wireframe — Dashboard (Desktop)

Figure 2. Structural change from today's `DashboardView`: the computed-but-unrendered Action Queue (Section 03, problem 2) now leads the page, above the KPI band. Everything else — KPI cards, category breakdown, sales chart, hot parts — is preserved but re-ordered by urgency, not by decoration.



Annotation: Action Queue band directly renders `buildActionQueue()`'s existing output (max 4 items, tone-colored: danger/warning/info/success) — same data source as today, new position and visual treatment only.

SECTION 12

12 Wireframe — Dashboard (Mobile / Narrow)

Figure 3. Below 768px, the icon rail collapses to a bottom tab bar (5 primary destinations + "More"), the KPI band becomes a horizontally-scrollable single row, and the Action Queue remains the very first element — non-negotiable across breakpoints per the shop-floor time-boxed constraint noted in Section 02.

FIGURE 3 — Dashboard, Mobile ≤428px — Bottom tab bar · single-column stack · swipeable KPI strip

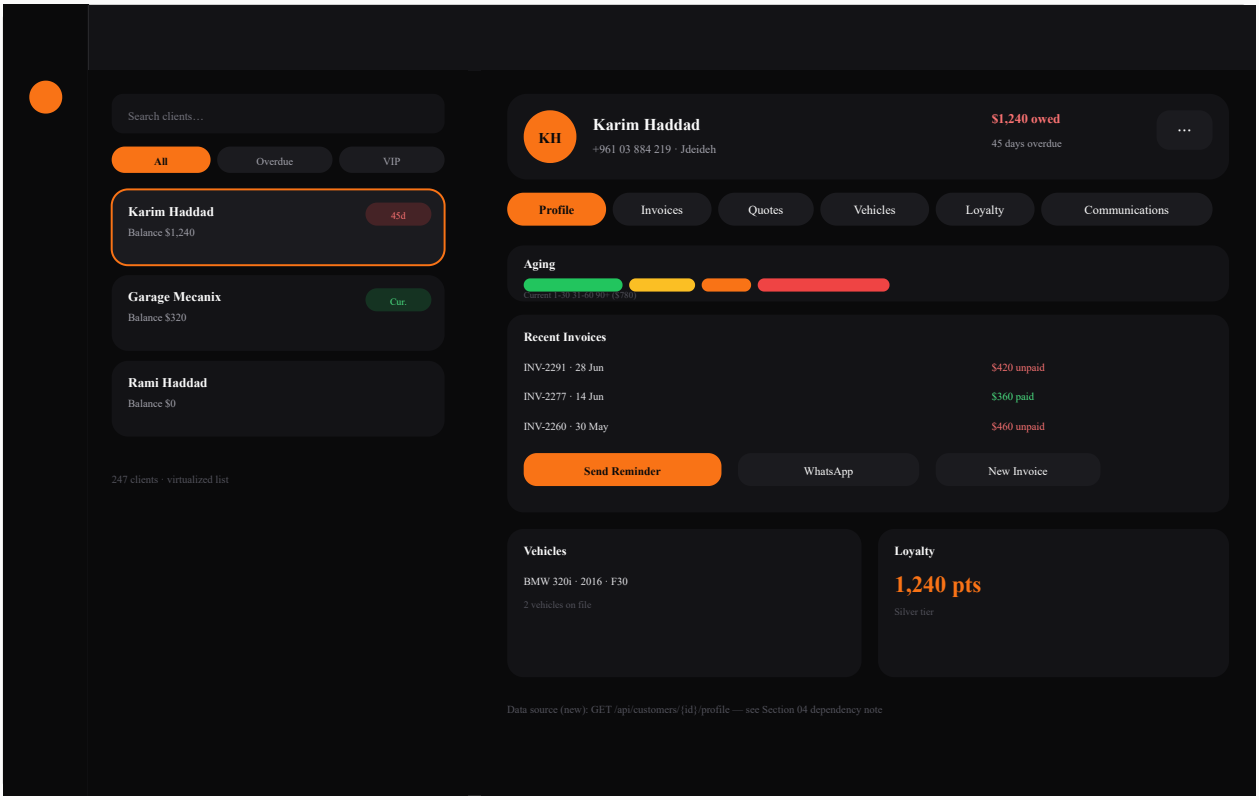
Annotation: Touch targets on the bottom tab bar are 44×44pt minimum (Section 15). The "More" tab opens the same 8-destination structure as the desktop rail flyout, presented as a full-screen sheet rather than a hover flyout.

SECTION 13

13 Wireframe — Client Workspace (Desktop)

Figure 4. Replaces Contacts (`contacts-view.js`), Customer Aging (`customer-aging-view.js`), and the customer-facing portions of Invoices, Quotes, Part Requests, Loyalty, and WhatsApp with one two-pane workspace: client list (left) and full client profile (right), organized as tabs per Section 05's progressive-disclosure figure.

FIGURE 4 — Client Workspace, Desktop ≥1280px — List + profile split-pane · opens as drawer from POS (Flow 03)



Annotation: This same component renders as the side-drawer referenced in Flow 03 (Section 08) — opened over any screen without navigating away, so POS/invoice state underneath is preserved.

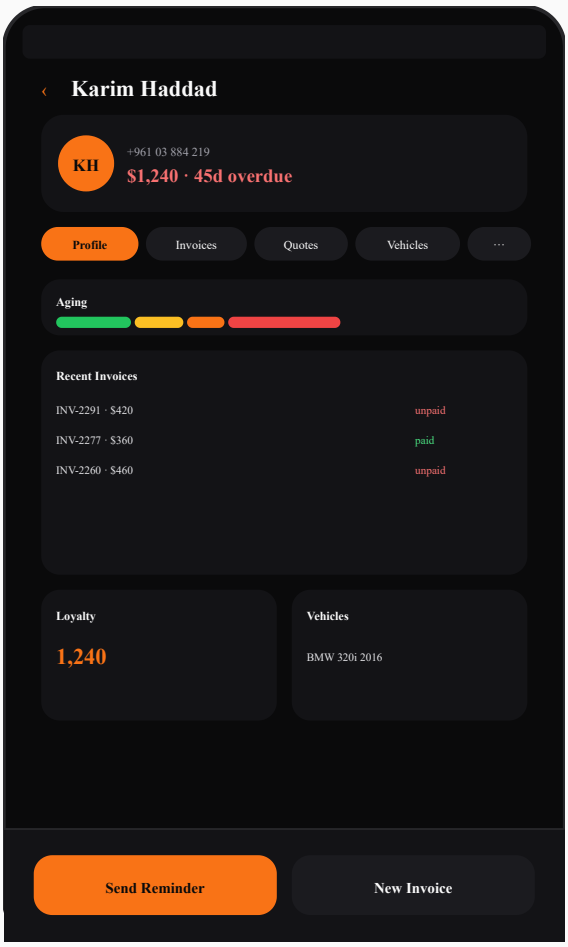
SECTION 14

14 Wireframe — Client Workspace (Mobile / Narrow)

Figure 5. The split-pane collapses to a single-column stack: list first, tapping a client pushes a full-screen profile with the same tab set rendered as a horizontally-scrollable chip row instead of a fixed tab strip.

FIGURE 5 — Client Workspace, Mobile ≤428px

List → push profile · sticky action bar



Annotation: The two most common counter actions (Send Reminder, New Invoice) are pinned to a sticky bottom bar so they remain reachable without scrolling — consistent with the touch-target and one-thumb-reach rules in Section 15.

SECTION 15

15 Accessibility Compliance Plan — WCAG 2.2 AA

Every wireframe in this report — and every component spec in Reports 02–06 — must satisfy the following, verified against the actual Ignition token values, not generic guidance.

15.1 Contrast Strategy

Pairing	Ratio	Result	Use
#fafafa on #0a0a0b	19.6:1	AAA	Primary text, dark surfaces
#a1a1aa on #0a0a0b	7.8:1	AAA	Secondary text
#f97316 on #0a0a0b	6.2:1	AA (LARGE + NORMAL)	Accent text, active nav
#0a0a0b on #f97316 (CTA button)	6.2:1	AA	Primary buttons — dark text on orange, not white on orange
#ef4444 on #0a0a0b	4.9:1	AA LARGE ONLY	Danger badges must pair with icon + bold weight, not color alone
#fbbf24 on #0a0a0b	10.8:1	AAA	Warning badges
#18181b on #ffffff (light mode body)	16.1:1	AAA	Light-mode primary text

Rule enforced in every wireframe above: the danger accent (#ef4444) is never the sole signal — every danger/overdue state pairs color with a text label ("45d overdue", "unpaid") or icon, satisfying WCAG 1.4.1 (Use of Color). This is visible in Figures 4–5 where the aging badge always carries a numeric day-count, not just a red dot.

15.2 Keyboard Navigation Map

Zone	Behavior
Icon rail	Arrow Up/Down moves focus between destinations; Enter/Space opens flyout; Escape closes flyout and returns focus to the rail icon.
Command palette	Ctrl+K / ⌘K opens from anywhere (fixing the dead affordance in `Topbar.js`, Section 03 problem 6); Arrow Up/Down cycles results; Enter navigates; Escape closes and returns focus to the element that had it before open.
Client Workspace drawer	Opens with focus moved to the drawer's close button; Tab is trapped within the drawer while open (focus trap); Escape closes and returns focus to the triggering element (e.g. the POS customer field from Flow 03).
Data tables	Sortable column headers are real buttons (already true in `DataTable.js`, lines 74–83) reachable by Tab, toggled by Enter/Space; row actions reachable via Tab after each row without a mouse.
Tab strips (Client Workspace profile tabs)	Left/Right arrow moves between tabs per the ARIA Tabs pattern; Tab key moves out of the tab list into the active panel.

15.3 Focus Order, Landmarks, Motion, Touch

15.3 Focus Order

Reading/focus order on every screen: skip-link → rail → topbar (search, then actions) → page heading → primary content region → secondary panels, left to right, top to bottom, matching visual order exactly. Because Figure 2 promotes the Action Queue above the KPI band, its focus order is promoted identically — a screen reader user reaches the day's urgent items before the decorative KPI band, not after it, matching the sighted-user priority stated in Flow 01 (Section 06).

15.4 Screen Reader Landmarks

Landmark	Role / Label
Icon rail	`nav`, `aria-label="Primary navigation"`
Topbar search	`search` landmark, input `aria-label="Search screens, clients, invoices, or parts"`
Main content	`main`, one per view, page `h1` as first heading inside it
Action Queue band	`region`, `aria-label="Action queue — items requiring attention"`, each item announced with its tone ("Danger: Receivables, 6 unpaid, $4,120")
Client Workspace drawer	`dialog`, `aria-modal="true"`, `aria-labelledby` pointing to the client name heading
Data tables	Native `table/th scope="col"` retained (already correct in `DataTable.js`) — no ARIA grid re-implementation needed

15.5 Reduced Motion

All chart line-draw animations, drawer slide-ins, and KPI count-up transitions respect `prefers-reduced-motion: reduce` by dropping to instant-state changes with no easing. The sales chart in Figures 2–3 renders its final path immediately rather than animating point-by-point when this preference is set. This applies identically to WPF (Storyboard duration 0) and React Native (Reanimated disabled) per platform, to be specified precisely in Reports 05–06.

15.6 Touch Target Sizing

Element	Minimum size	Note
Bottom tab bar icons (mobile)	`44×44pt`	Figure 3 — rail collapses to 5 tabs, each full 44pt hit target even though the visual icon is smaller
Sticky action bar buttons (Figure 5)	`44×44pt (height 32px visual + 6px padding)`	"Send Reminder" / "New Invoice" — the two highest-frequency mobile counter actions
Table row action buttons	`32×32px desktop / 44×44pt touch`	Larger target on touch input via CSS pointer media query, not two separate components
Client list rows (Figure 4/5)	`Full-row tap target, min 44pt height`	Entire row is the hit area, not just the client name text

SECTION 16

16 Cognitive Load & Progressive Disclosure

The current dashboard's core problem (Section 03, problems 2 and 7) is not missing data — it is undifferentiated data. Ignition's disclosure rules make that differentiation explicit and consistent across both the Dashboard and Client Workspace.

16.1 Dashboard — Visible by Default vs. On Demand

Visible immediately (no interaction)	Revealed on demand
Action Queue (max 4 items, tone-sorted danger→success)	Full unpaid-transactions list (link from Receivables card)
6–8 KPI cards, single row	Per-KPI trend detail / historical comparison (click-through)
This-week sales chart, current totals	12-month profit history (already exists in `InvoicesView` but is buried below the fold today — relocated to an on-demand drill-in, not dashboard-default)
Hot Parts (top 5), Quick Actions grid	Full category breakdown table, full inventory snapshot donut legend

This directly resolves Section 03 problem 7 (Sales/Profit Today repeating 2–3 times): under the new rule, a value appears in exactly one default-visible location (the KPI band) and any repetition (e.g., the old "bottom stat bar") is removed rather than duplicated.

16.2 Client Workspace — Visible by Default vs. On Demand

Visible immediately (Profile tab)	Revealed on demand (other tabs)
Identity, phone, balance, aging bar	Full invoice history with filters (Invoices tab)
Last 3 invoices with paid/unpaid status	Full quote history and draft builder (Quotes tab)
Primary quick actions: Send Reminder, WhatsApp, New Invoice	Active + historical part requests (Part Requests tab)
Vehicle count + most recent vehicle	Full vehicle list with VIN/plate detail (Vehicles tab)
Loyalty points total + tier	Full points ledger, redemption history (Loyalty tab)
—	Full communication timeline across WhatsApp/SMS/notes (Communications tab)

16.3 Principles Applied

- **Urgency before volume:** exceptions (Action Queue, overdue badges) always render before comprehensive listings, on both Dashboard and Client Workspace.
- **One canonical location per fact:** a given number (balance, sales total) has exactly one default-visible home; drill-ins reference it, they don't duplicate it in a different format.
- **Context-preserving disclosure:** the Client Workspace opens as a drawer over POS (Flow 03) specifically so "revealing more" never means "losing where you were" — this is a cognitive-load rule as much as a technical one.
- **Task-scoped navigation over feature-scoped navigation:** the 8-destination IA (Section 04) groups by what a persona is trying to do (serve a client, manage inventory) rather than by internal feature name, reducing the search cost of "where do I click" that the current 70-screen flat structure imposes.

HANDOFF

This report defines flows, IA, wireframe structure, and accessibility/disclosure rules only. Visual treatment (exact component styling, spacing scale, elevation system) is owned by dev-graphic-designer in Report 02, built on these wireframes. Implementation is routed to dev-web-react, dev-

desktop-wpf, and dev-mobile-rn only after Ralph reviews and approves this flow — per the Design Change Workflow, no code has been written as part of this report.